

Grievance Redressal Survey Results

S. Naveera Fatima Rizvi

Source= Fake data inspired from [BISP Survey](#)

 Questionnaire

```
In [60]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('ggplot')
from texttable import Texttable
import plotly.io as pio
pio.renderers.default= 'notebook_connected'
import plotly.express as px
import plotly.graph_objects as go
```

```
In [61]: # Load data with a separate dataframe for seperate sheets

# Load sheet with responses e.g Q1:a, Q2b
df_raw_responses= pd.read_excel("Clean_Grievance_Sample.xlsx", sheet_name='Responses')

#Load sheet with keys to map responses to labels e.g a --> Yes, b ---> No
df_raw_keys= pd.read_excel("Clean_Grievance_Sample.xlsx", sheet_name="Keys")
df_raw_keys= df_raw_keys.set_index("Options")

#Load sheet with question labels mapping to numbers e.g.
df_raw_questions= pd.read_excel("Clean_Grievance_Sample.xlsx", sheet_name="Questions")
```

```
In [62]: #Responses
df_raw_responses.head()
```

```
Out[62]:
```

	PROVINCE	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8
0	AJK/GB	b	a	a	a	a	NaN	NaN	NaN
1	KP	b	a	b	NaN	NaN	b	a	NaN
2	KP	a	c	b	NaN	NaN	b	b	NaN
3	KP	e	b	b	NaN	NaN	b	c	NaN
4	Balochistan	b	a	a	a	a	NaN	NaN	NaN

```
In [63]: # Answer Keys
df_raw_keys
```

```
Out[63]:
```

		Q1	Q2	Q3	Q4	Q5	Q6		Q7		Q8
Options											
a	Quality of Service Issues	In Person	Yes	Yes	Yes	Yes		Lack of time		Complaint resolved and I am satisfied	
b	Payment Issues	Phone	No	No	No	No		Language barrier		Complaint resolved but I am not satisfied	

Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8

Options

c	Documentation Issues	Other	NaN	NaN	NaN	NaN	Didn't feel it would make a difference	Complaint not resolved
d	Eligibility Issues	NaN	NaN	NaN	NaN	NaN	Other	Other
e	Other	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
In [64]: # Questions
df_raw_questions
```

```
Out[64]:
```

	Number	Question
0	Q1	What was the reason behind your complaint?
1	Q2	How did you lodge a complaint?
2	Q3	Did anyone contact you?
3	Q4	Was your complaint resolved?
4	Q5	Are you satisfied with the resolution?
5	Q6	Did you complain again?
6	Q7	Why didn't you complain again?
7	Q8	What was the outcome of complaining again?

```
In [65]: # Create a nested dictionary mapping question numbers to answers

dict_map = {} # Create empty dictionary for question number: {answer:option label} pairs
for question_number in df_raw_keys.columns:
    dict_answers = {} # For each question, create an empty dictionary for the relevant answers
    df_answer_labels = df_raw_keys[question_number].dropna() # Drop missing values (when no answer)
    for option in df_answer_labels.index:
        dict_answers[option] = df_answer_labels[option]
    dict_map[question_number] = dict_answers

df = df_raw_responses.replace(dict_map) # Use the newly created dictionary to map relevant answers

new_colnames = list(df_raw_questions["Number"]+": "+df_raw_questions["Question"])
new_colnames.insert(0,"Province")
df.columns = new_colnames

df.head()
```

```
Out[65]:
```

	Province	Q1: What was the reason behind your complaint?	Q2: How did you lodge a complaint?	Q3: Did anyone contact you?	Q4: Was your complaint resolved?	Q5: Are you satisfied with the resolution?	Q6: Did you complain again?	Q7: Why didn't you complain again?	Q8: What was the outcome of complaining again?
0	AJK/GB	Payment Issues	In Person	Yes	Yes	Yes	NaN	NaN	NaN
1	KP	Payment Issues	In Person	No	NaN	NaN	No	Lack of time	NaN
2	KP	Quality of Service Issues	Other	No	NaN	NaN	No	Language barrier	NaN

		Q1: What was the reason behind your complaint?	Q2: How did you lodge a complaint?	Q3: Did anyone contact you?	Q4: Was your complaint resolved?	Q5: Are you satisfied with the resolution?	Q6: Did you complain again?	Q7: Why didn't you complain again?	Q8: What was the outcome of complaining again?
3	KP	Other	Phone	No	NaN	NaN	No	Didn't feel it would make a difference	NaN
4	Balochistan	Payment Issues	In Person	Yes	Yes	Yes	NaN	NaN	NaN

In [66]:

```
# Create a dataframe for lat_long data
lat_long= pd.DataFrame({
    'Province': ["Punjab", "Sindh", "KP", "Balochistan", "AJK/GB"],
    'Lat': [31.1704, 25.8943, 34.9526, 28.4907, 34.8641],
    'Long': [72.7097, 68.5247, 72.3311, 65.0958, 74.4232]
}).set_index("Province")

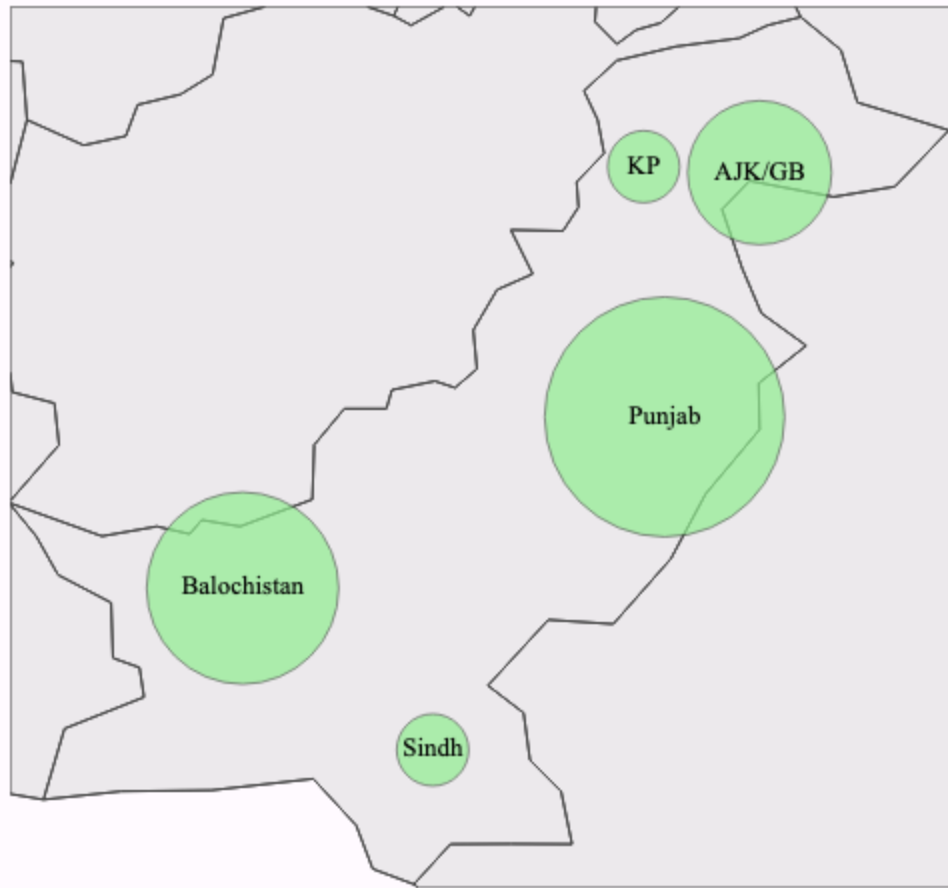
# Join Province Counts with Lat Long data
data_map = df["Province"].value_counts().to_frame()
data_map = data_map.join(lat_long)
data_map = data_map.reset_index()
data_map.rename(columns={'index': 'Region'}, inplace=True)

# Plot Bubble map
fig = go.Figure()
fig.add_trace(go.Scattergeo(
    lat=data_map["Lat"], lon=data_map["Long"],
    marker = dict(
        size = data_map["Province"]*12,
        color='lightgreen',
        line_color='rgb(40,40,40)',
        line_width=0.7),
    showlegend=False)
)

fig.add_trace(go.Scattergeo(
    lat=data_map["Lat"], lon=data_map["Long"],
    text=data_map["Region"],
    textposition="middle center",
    textfont=dict(size =13, color='black', family='serif'),
    mode='text',
    showlegend=False)
)

fig.update_layout(
    title_text='<b>Bubble Map for Responses',
    titlefont=dict(size =30, color='black', family='serif'),
    title_x=0.5,
    geo=dict(scope='asia', lataxis_range=[23.9,37.5], lonaxis_range=[60.9, 78],
    landcolor = 'rgba(217, 217, 217,0.5)',
    showcountries = True,
    showcoastlines=True,
    bgcolor='rgb(255,250,255)'),
    paper_bgcolor='rgb(255,250,255)',
    margin={"r":10,"t":50,"l":10,"b":10}
)
fig.show("png")
```

Bubble Map for Responses



In [67]:

```
def plot_pie(data,color_list,title,ax_num):
    data.value_counts().plot(
        kind='pie',
        ax=ax[ax_num],
        colors= color_list,
        autopct='%1.1f%%',
        textprops={'fontsize': 12,'color':'darkblue','fontweight':550,'family':'serif'},
        wedgeprops = {'width':0.6,'linewidth':3, 'edgecolor':(1,1,1,1)})

    ax[ax_num].set_title(title,
        loc='center', fontsize=25, fontweight="bold",family='serif')
    ax[ax_num].set_ylabel("")
    ax[ax_num].tick_params(axis='x', labelsiz=25)
    ax[ax_num].tick_params(axis='y', labelsiz=25)
```

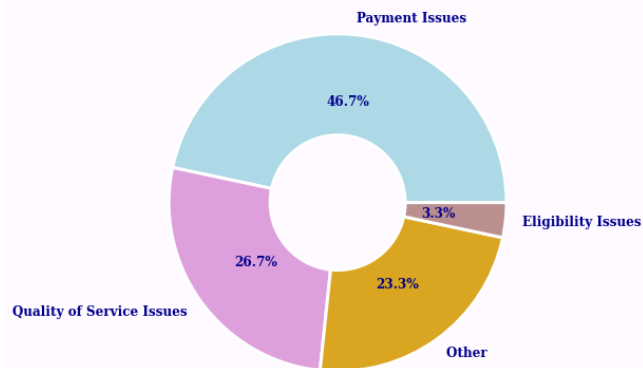
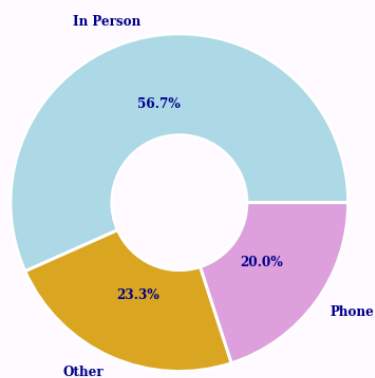
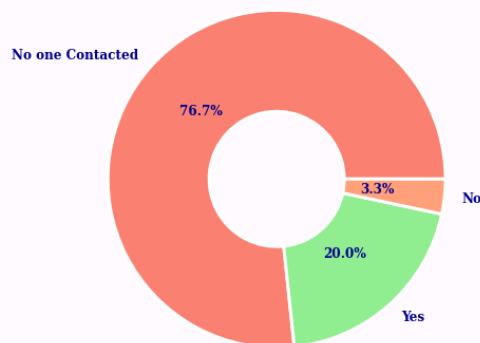
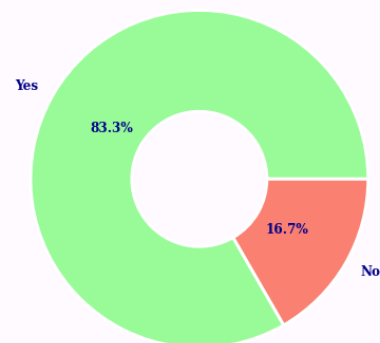
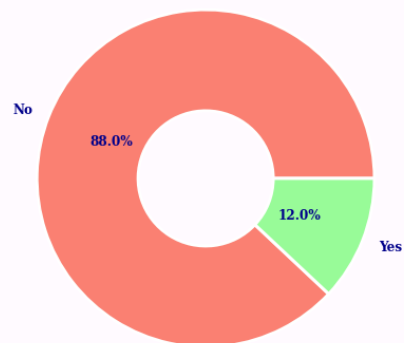
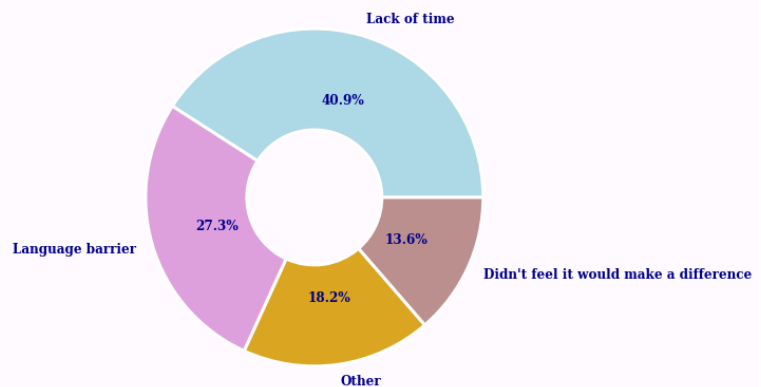
In [68]:

```
fig,ax= plt.subplots(3,2,figsize=(20,25))

ax = ax.flatten()

plot_pie(df["Q1: What was the reason behind your complaint?"],['lightblue', 'plum','goldenrod'])
plot_pie(df["Q2: How did you lodge a complaint?"],['lightblue','goldenrod','plum'],'Complete')
plot_pie(df["Q4: Was your complaint resolved?"].fillna("No one Contacted"),['salmon','lightblue'],'Complete')
plot_pie(df["Q5: Are you satisfied with the resolution?"].dropna(),['palegreen','salmon'],'Complete')
plot_pie(df["Q6: Did you complain again?"].dropna(),['salmon', 'palegreen'],'Did you complain again?')
plot_pie(df["Q7: Why didn't you complain again?"].dropna(),['lightblue', 'plum','goldenrod'],'Complete')

fig.set_facecolor(tuple(np.array([255, 250, 255]) / 255))
plt.show()
```

Complaint Reasons**Complaint Source****Complaint Resolved?****Satisfied with Resolution?****Did you complain again?****Reasons for Not Complaining Again**

In [69]:

```
# Define function for plotting barchars
def plot_bars(col, barcolors):
    """
    This function plots a barchart for a given column
    input: Relevant column and colors for barcharts
    output: piechart

    """
    data_bar = pd.crosstab(df[col], df["Province"], dropna=False)
    data_bar["Total"] = data_bar.sum(axis=1)

    # Convert numbers to percentages
    data_bar = data_bar.apply(lambda r: r*100/r.sum(), axis=0)
    data_bar = data_bar.round(decimals=1)

    # Reorder columns
    data_bar = data_bar.loc[:, ["Punjab", "Sindh", "KP", "Balochistan", "AJK/GB", "Total"]]
```

```

#Plot barchart
fig,ax= plt.subplots(figsize=(48,20))
data_bar.T.plot.bar(
    ax=ax, rot=0, legend=False,
    width=0.8,color=barcolors, edgecolor='k')

# Add this loop to add the annotations
for p in ax.patches:
    width = p.get_width()
    height = p.get_height()
    if height>0:
        x, y = p.get_xy()
        ax.annotate(
            height, (x + width/2, height+1.02),ha='center',
            fontsize=25,fontweight="bold", color='darkblue')

ax.legend(loc='best',prop={'size': 30})
ax.set_title("\n"+data_bar.index.name+"\n", loc='center', fontsize=50, fontweight="bold")
ax.set_xlabel('Region',fontsize=40, fontweight="bold",family='serif')
ax.set_ylabel('Percentage',fontsize=40, fontweight="bold",family='serif')
ax.tick_params(axis='x', labelsiz=25)
ax.tick_params(axis='y', labelsiz=25)

fig.set_facecolor(tuple(np.array([255, 250, 255]) / 255))
fig.tight_layout()

```

In [70]:

```

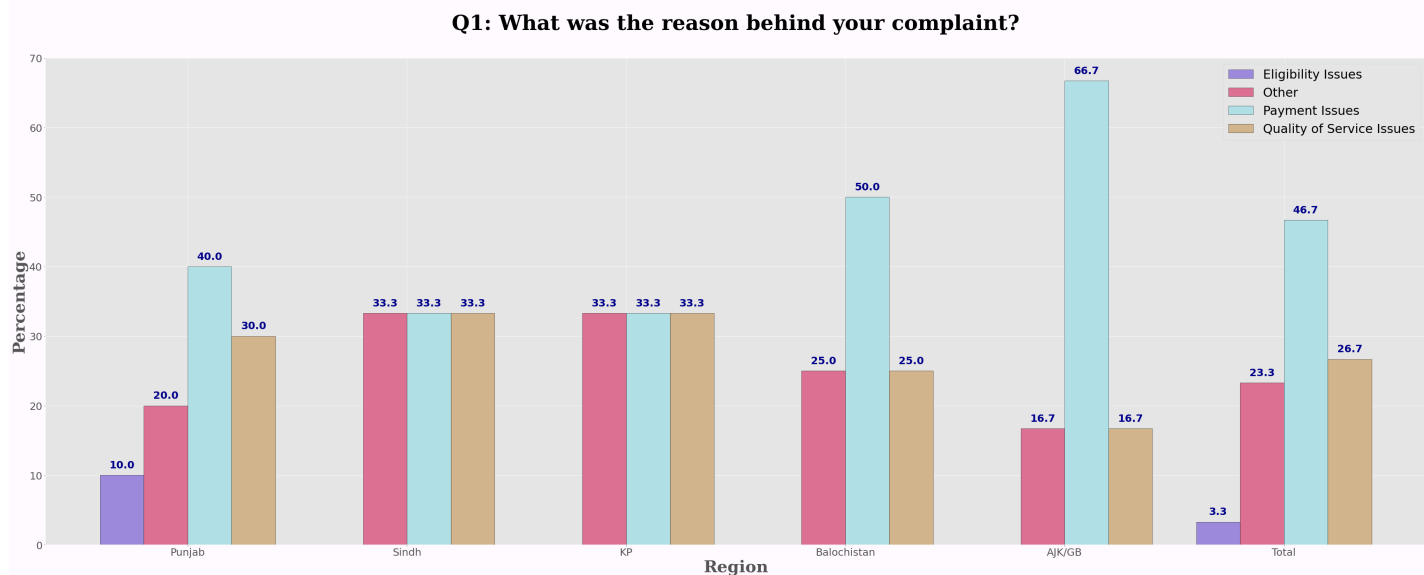
# Define colors for Barcharts

# Red and Green colors for Yes/No Questions
col_yn={'Yes':(0.1,0.7,0.1,0.8),'No':(0.7,0.1,0.1,0.8),}

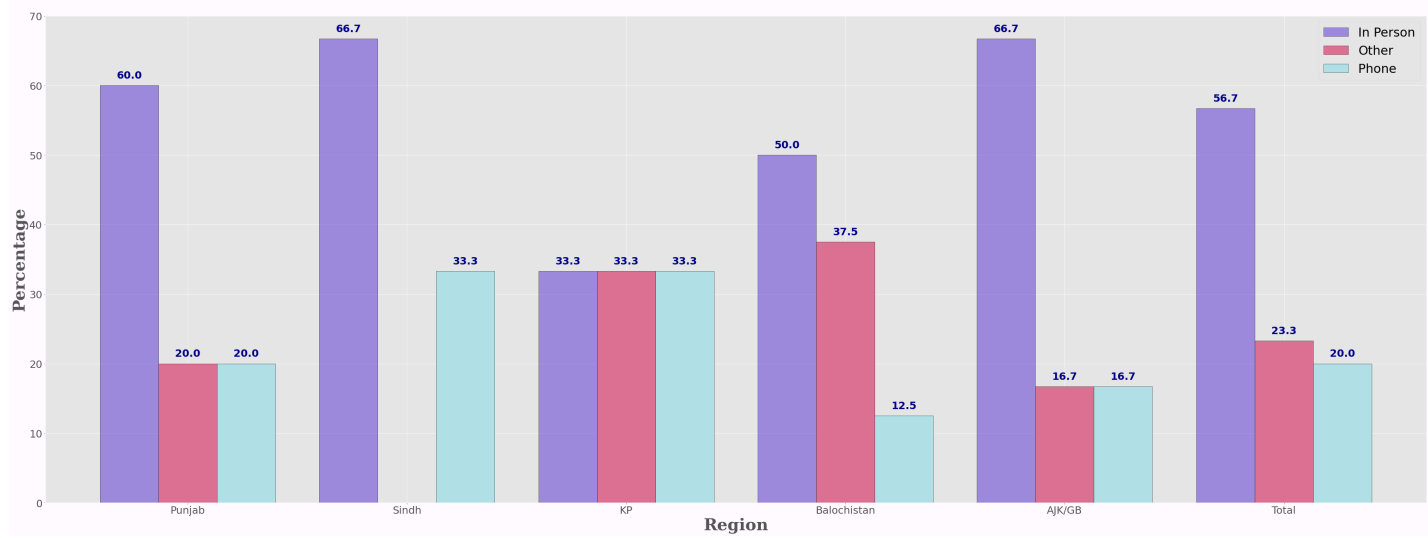
# Colors for other questions
clr=[(0.2,0,0.8,0.4),'palevioletred','powderblue','tan']

# Plot Bar charts
for i in range(1,5):
    if i > 2:
        # Yes and No Questions
        plotBars(df.columns[i],col_yn)
    else:
        # All other questions
        plotBars(df.columns[i],clr)

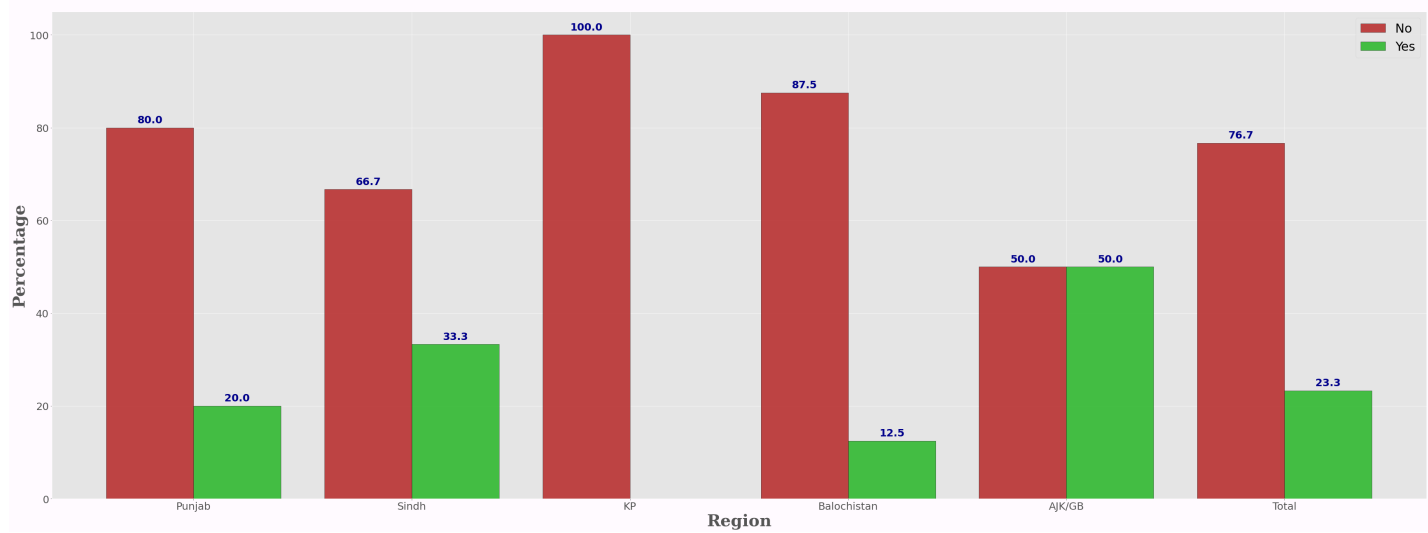
```



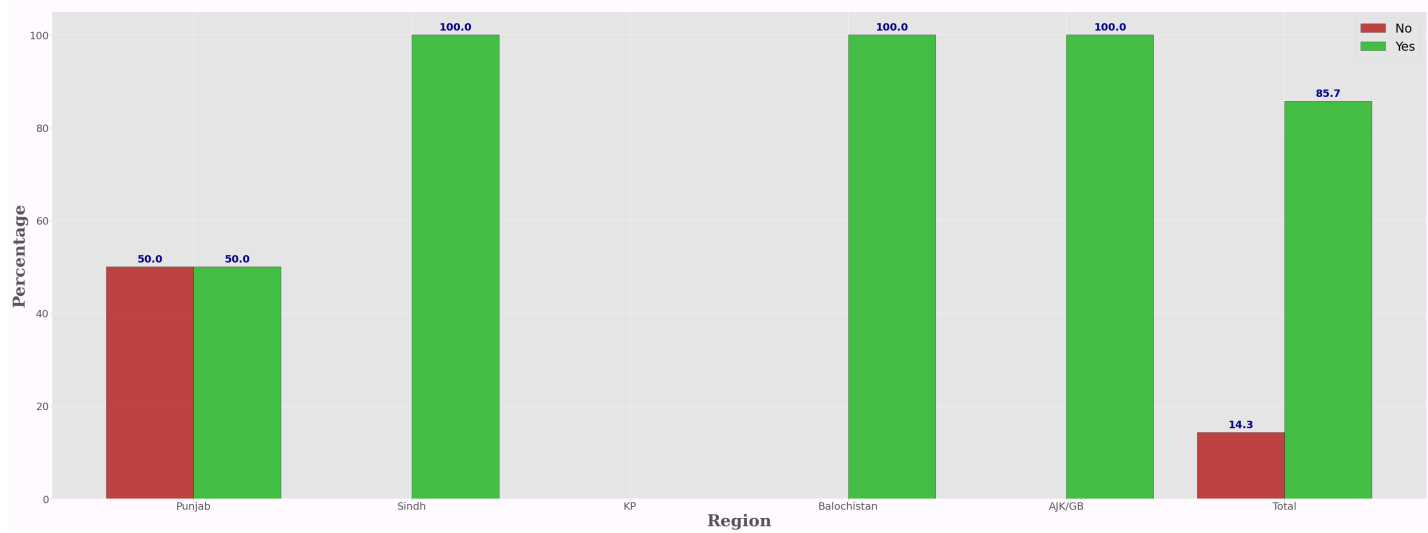
Q2: How did you lodge a complaint?



Q3: Did anyone contact you?



Q4: Was your complaint resolved?



```
In [ ]: ! jupyter nbconvert --to webpdf Grievance_redressal_sampledata.ipynb
```

```
In [ ]:
```